Automated Seat Belt Detection – A Deep Learning Approach

One of the projects I worked on was **Automated Seat Belt Detection using Deep Learning**.

The main objective of this project was to develop a **real-time computer vision system that can automatically detect whether a person is wearing a seat belt or not**. The motivation behind the project was road safety, because manually monitoring seat belt violations is difficult, especially when there are multiple vehicles or passengers. So, I wanted to build an automated system that could analyze images or video frames and identify seat belt violations using deep learning.

For this project, I used **YOLO-based object detection models**, specifically **YOLOv8n, YOLOv8s, and YOLOv5**. I used these models because YOLO is a single-stage object detection approach and is well suited for real-time applications.

The overall workflow of my project was:

Input image or video → preprocessing → YOLO model → object detection → seat belt classification → final prediction.

First, I prepared the images required for training the object detection models. The images contain people inside vehicles, and the objective is to identify the seat belt-related regions and determine whether the person is complying with the seat belt requirement.

One of the important challenges I considered was **low-light conditions**. In real-world scenarios, cameras may capture images in poor lighting, especially at night or inside vehicles. In low-light images, the contrast between the seat belt and the person's clothes or the vehicle interior can become very low. This makes detection more difficult.

To address this problem, I used **CLAHE, which stands for Contrast Limited Adaptive Histogram Equalization**, as an image preprocessing technique.

The basic idea behind CLAHE is to improve the local contrast of an image. Instead of enhancing the contrast of the entire image uniformly, CLAHE works on small regions of the image and improves the visibility of important details while limiting excessive contrast enhancement.

So, my preprocessing pipeline was designed to make the relevant visual features more distinguishable, particularly under low-light conditions.

After preprocessing, I trained different YOLO models.

The first model I experimented with was **YOLOv5**. I used it as one of the models for comparison, particularly to evaluate its performance in scenarios involving multiple people.

Then I worked with **YOLOv8**, using both the **YOLOv8n and YOLOv8s variants**.

The difference between these two variants is mainly the trade-off between model size, speed, and accuracy. YOLOv8n is the nano version, so it is lightweight and designed for faster inference. YOLOv8s is slightly larger and has more capacity, so it can generally provide better detection performance at the cost of some additional computation.

During inference, the image is passed through the YOLO model.

Internally, YOLOv8 has three major parts: the **backbone, neck, and detection head**.

The backbone extracts useful visual features from the input image. For example, it learns features such as edges, shapes, textures, and other patterns.

Then the neck combines features from different levels of the network. This is important because objects can appear at different sizes. The resulting feature maps contain useful information about the objects present in different regions of the image.

Finally, the detection head uses these feature maps to predict the **bounding box, class, and confidence score** for the detected object.

So, for a detected seat belt, the model essentially gives me information about **where the object is, what class it belongs to, and how confident the model is about the prediction**.

One reason YOLO is suitable for this project is that it is a **single-stage object detector**. Instead of first generating object proposals and then separately classifying them, YOLO directly predicts the object locations and classes in a forward pass. This makes it suitable for real-time applications.

I then evaluated the different models based on their detection performance and inference speed.

The **YOLOv8s model achieved 90.6% accuracy** in my experiments. This showed that the larger YOLOv8s model was able to provide strong detection performance.

I also evaluated **YOLOv5 for multi-person detection**, where it achieved approximately **90% accuracy**.

At the same time, when real-time performance was important, **YOLOv8n was particularly useful because it achieved an inference time of around 3.5**

milliseconds per frame in my experiments.

Therefore, there was a clear trade-off between accuracy and computational speed.

YOLOv8s provided the best accuracy among the models I tested, while YOLOv8n provided very fast inference because it is a lightweight model.

The final idea of the project is that a camera can continuously capture frames, the model can process those frames, and the system can identify seat belt violations automatically. This makes the approach useful for applications such as intelligent traffic monitoring and automated road-safety systems.

The main things I learned from this project were **deep learning-based object detection, YOLO architecture, image preprocessing, model training and evaluation, and the importance of balancing accuracy with inference speed in real-time computer vision applications**.

If I were to improve the project further, I would focus on increasing the diversity of the training data, improving performance under different lighting and camera conditions, and optimizing the model further for deployment on edge devices.

So, in summary, I developed an **automated seat belt detection system using YOLOv5 and YOLOv8 variants, incorporated CLAHE preprocessing to improve robustness in low-light conditions, and compared the models based on accuracy and inference speed. The best accuracy I obtained was 90.6% with YOLOv8s, while YOLOv8n achieved approximately 3.5 milliseconds per frame for fast inference.**

Alert Mechanism

In addition to detecting seat belt violations, I also implemented an **alert mechanism using Twilio** to make the system more practical.

When the model detects that the driver is not wearing a seat belt, the detection result is passed to the alert module. The system then triggers a notification through the **Twilio API** to alert the driver.

So the complete system is not limited to simply displaying a bounding box around the detected violation. It also takes an action when a violation is identified.

The flow is:

Camera/Input Frame → YOLO Detection → Seat Belt Violation Detected → Alert Trigger → Twilio Notification → Driver is Notified.

This makes the project more suitable for a real-world road-safety application because the system can automatically detect a violation and notify the concerned person instead of requiring continuous manual monitoring.

For example, if the driver is detected without a seat belt, the system can trigger a Twilio-based alert informing the driver that the seat belt is not being worn.

I considered this alert mechanism as an additional layer on top of the computer vision model. The YOLO model is responsible for the **detection**, while Twilio is responsible for the **communication/notification** after a violation is detected.

"I integrated Twilio's Programmable Messaging API with the YOLO-based detection pipeline. YOLOv8 first processes the camera frame and detects whether the driver is wearing a seat belt. When a seat-belt violation is detected, my Python application triggers the Twilio API. The application sends an authenticated request containing the recipient number and alert message. Twilio then processes the request through its messaging infrastructure and delivers the SMS to the driver's phone. So YOLO handles the AI-based detection, while Twilio handles the communication and notification part."

TELL ME ABOUT THE DATASET AND PREPROCESSING.

For the seat belt detection project, the first step was preparing the image dataset for object detection.

Since YOLO is a supervised learning model, the images need to be accompanied by annotations that tell the model where the target object is located. For each relevant object in an image, the annotation contains the class information along with the bounding-box coordinates.

The images were then divided into training and validation data so that the model could learn from the training images and be evaluated on images that it had not seen during training.

One of the main challenges in this project was that seat belt detection can become difficult under poor lighting conditions. A seat belt may have low contrast with the person's clothes or the vehicle interior, so its visual features may not be clearly visible.

To address this, I used CLAHE, or Contrast Limited Adaptive Histogram Equalization, as a preprocessing technique.

CLAHE improves local contrast rather than simply increasing the brightness of the entire image. The image is divided into small regions, called tiles, and histogram equalization is applied locally to those regions. The contrast enhancement is then limited using a clip limit so that noise is not excessively amplified.

This helps reveal important details in low-light images while avoiding the excessive contrast enhancement that ordinary histogram equalization can sometimes produce.

After preprocessing, the images and their corresponding annotations were provided to the YOLO models for training.

An important point is that preprocessing should not change the object annotations incorrectly. If an operation changes the spatial geometry of the image, such as resizing or cropping, the corresponding bounding-box coordinates also need to be adjusted.

So the overall data-processing pipeline was:

Image dataset → Annotation → Preprocessing with CLAHE → Dataset preparation → YOLO training.

WHY DID YOU CHOOSE YOLO? WHY YOLOV5 AND YOLOV8?

I chose YOLO because my project required **real-time seat belt detection** from images or video.

There are different approaches for object detection, including two-stage detectors such as Faster R-CNN and single-stage detectors such as YOLO. Since my application is focused on real-time detection, I preferred a single-stage detector because it can directly predict the object location and class in a single forward pass, which makes it faster.

Another reason for choosing YOLO is that it performs both localization and classification as part of the same detection pipeline. This makes it suitable for detecting seat belts from video frames.

I experimented with both YOLOv5 and YOLOv8 to compare their performance rather than relying on only one model.

I used YOLOv8 in two variants: YOLOv8n and YOLOv8s.

YOLOv8n is the nano version, which is smaller and computationally lighter. I used it to investigate whether the system could achieve very fast inference while maintaining reasonable detection performance.

YOLOv8s is a slightly larger model with more capacity. Because it has more parameters and computational capability than the nano version, it can generally learn more complex visual patterns and provide better detection performance, although it requires more computation.

I also used YOLOv5 as another baseline for comparison, particularly for evaluating its performance in multi-person scenarios.

The comparison helped me understand the trade-off between **accuracy and inference speed**.

In my experiments, YOLOv8s achieved 90.6% accuracy, while YOLOv5 achieved around 90% accuracy in the multi-person scenario. On the other hand, YOLOv8n achieved approximately 3.5 milliseconds per frame, making it particularly attractive when real-time speed is important.

Therefore, I did not select a model only based on accuracy. I compared the models based on both **detection performance and computational speed**, because both are important for a real-time seat belt detection system.

EXPLAIN THE YOLOV8 ARCHITECTURE USED IN YOUR PROJECT.

In my project, I used YOLOv8 for real-time seat belt detection.

YOLOv8 is a single-stage object detection model. Its main architecture can be understood in three major parts: the **Backbone, Neck, and Detection Head**.

The overall flow is:

Input Image → Backbone → Neck → Detection Head → Final Detection.

First, the input image is given to the Backbone.

The main purpose of the Backbone is **feature extraction**. It takes the raw image and gradually converts it into feature representations.

In the initial layers, the network learns relatively simple features such as edges, corners, lines, and textures. As we move deeper into the network, the features become more

meaningful and represent higher-level patterns such as parts of a person, vehicle interior, or the shape and appearance of a seat belt.

One important component in the YOLOv8 backbone is the **C2f module**.

The C2f module helps the network efficiently process and combine features while maintaining good gradient flow during training. In simple terms, it allows the network to reuse useful information from different layers instead of completely discarding earlier features.

Another important component is **SPPF, or Spatial Pyramid Pooling Fast**.

SPPF helps the network capture information from different spatial scales. This is useful because objects can appear at different sizes in an image. For example, depending on the camera position, a seat belt or a person may occupy a different amount of the image.

After the Backbone extracts features, these features are passed to the **Neck**.

The Neck performs feature fusion. It combines information from different levels of the Backbone.

This is important because shallow layers contain more detailed spatial information, while deeper layers contain stronger semantic information.

For example, a shallow feature may contain information about a thin diagonal line, which can be useful for detecting a seat belt. A deeper feature may provide information that the line is associated with a person's torso.

The Neck combines these types of information and produces feature maps at different scales.

These feature maps are then given to the **Detection Head**.

The Detection Head converts the extracted features into actual predictions.

For every potential object, the Head predicts the **bounding box**, the **class**, and the corresponding confidence information.

The bounding box tells us where the object is located. It is represented using the center coordinates and the width and height of the box.

The classification part tells us what the detected object is. In my seat belt detection application, this corresponds to the classes defined in my dataset.

The model is also **anchor-free**, meaning that it does not depend on a predefined set of anchor boxes. Instead, it directly predicts the object bounding boxes from the feature

maps.

YOLOv8 also uses a **decoupled detection head**, where classification and bounding-box regression are handled separately. This allows the model to focus independently on identifying the object and locating it.

Finally, the predictions are post-processed to remove duplicate overlapping detections, producing the final bounding boxes and class predictions.

So, in simple terms, the Backbone answers:

"What visual features are present?"

The Neck answers:

"How can I combine useful information from different levels?"

And the Head answers:

"Where is the object, what is it, and how confident am I?"

For my seat belt detection project, this complete pipeline allows the system to take an input image or video frame and produce the location and class of the detected seat belt in real time.

WHAT DOES THE NECK DO IN YOLOV8?

After the Backbone extracts features from the image, the next component is the Neck.

The main purpose of the Neck is to **combine feature information from different levels of the Backbone** so that the detection head can detect objects of different sizes.

The reason this is necessary is that different layers of the Backbone contain different types of information.

The earlier layers have higher-resolution feature maps and preserve more spatial details. These details are useful for detecting small objects.

The deeper layers have lower-resolution feature maps, but they contain stronger semantic information. They help the model understand what the object actually represents.

For example, in my seat belt detection project, the seat belt can be a relatively small and thin object in the image. A high-resolution feature map can preserve details about the thin belt strap. At the same time, deeper features can provide contextual information about the person and the vehicle interior.

The Neck combines these different types of information.

YOLO-style architectures use feature fusion between different scales. Information from deeper layers is passed toward higher-resolution layers, and information from lower-resolution layers is also aggregated so that the network gets both detailed and semantic information.

The result is a set of **multi-scale feature maps** that contain richer information than the individual feature maps from the Backbone.

These feature maps are then passed to the Detection Head.

So, in simple terms, the Backbone extracts features, while the Neck **combines useful features from different levels and prepares them for detection**.

This multi-scale feature fusion is important because the same detection model may need to detect small objects, medium-sized objects, and large objects in the same image.

WHAT HAPPENS IN THE YOLOV8 DETECTION HEAD?

The Detection Head is the final major part of YOLOv8.

Its job is to take the multi-scale feature maps produced by the Neck and convert them into actual object detections.

In simple terms, the Head answers three main questions:

First, **where is the object?**

It predicts the bounding box coordinates of the object. The bounding box describes the object's position and size.

Second, **what is the object?**

It predicts the class of the detected object.

Third, **how confident is the model about the prediction?**

The model produces scores that are used to determine how reliable the detection is.

In my seat belt detection project, the Head receives feature maps containing information about patterns such as the person's body, clothing, and possible seat-belt regions. Based on the features learned during training, the Head predicts the location and class of the detected object.

YOLOv8 uses an **anchor-free detection approach**. This means that it does not depend on a predefined collection of anchor boxes. Instead, it directly predicts the bounding-box

information from the feature-map locations.

YOLOv8 also uses a **decoupled detection head**. The classification task and bounding-box regression task are handled through separate branches.

The classification branch focuses on answering:

"What object is this?"

while the regression branch focuses on:

"Where exactly is this object?"

Separating these two tasks helps the network learn the two objectives independently.

The Head performs these predictions at multiple feature-map scales. This allows YOLOv8 to detect objects of different sizes.

After the predictions are generated, post-processing is applied to remove redundant overlapping detections, resulting in the final bounding boxes and class predictions.

So the complete process is:

Feature maps from the Neck → bounding-box prediction + classification → filtering/post-processing → final detection.

YOU SAID THE HEAD PREDICTS X, Y, WIDTH AND HEIGHT. HOW DOES IT KNOW WHAT VALUES TO PREDICT?

The YOLOv8 Head does not know the correct bounding box initially. It learns to predict it during the training process.

During training, every image has a ground-truth annotation. The ground-truth annotation tells the model where the target object is located.

For example, suppose an image contains a seat belt. The annotation provides the correct bounding box around the seat belt.

The image is first passed through the Backbone and Neck, and the Detection Head produces a predicted bounding box.

The model then compares this predicted bounding box with the ground-truth bounding box.

If the prediction is different from the ground truth, an error is produced through the loss function.

The loss tells the model how far its prediction is from the desired prediction.

The error is then propagated backward through the network using backpropagation, and the weights of the neural network are updated using an optimizer.

This process happens repeatedly over many training images and many epochs.

For example, suppose the correct bounding box is:

x = 300

y = 200

width = 50

height = 100

but the model initially predicts something different.

The loss will be higher because the predicted box does not match the ground-truth box.

After many training iterations, the network gradually learns which visual features correspond to particular object locations and sizes.

So the model is not given a rule saying that a diagonal line means a seat belt. Instead, it learns this relationship from the training examples.

In my project, the same learning process allows the YOLO model to learn visual patterns associated with seat belts and then predict their locations when it receives an unseen image.

Therefore, the complete training process can be summarized as:

Image → Feature extraction → Prediction → Compare with ground truth → Calculate loss → Backpropagation → Update weights → Repeat.

WHAT LOSS FUNCTION DOES YOLOV8 USE?

YOLOv8 uses different loss components because object detection has more than one task.

The main tasks are:

1. Predicting the correct bounding box.
2. Predicting the correct class.

Therefore, the training loss considers both localization and classification.

For bounding-box prediction, YOLOv8 uses a combination of **IoU-based box loss and Distribution Focal Loss, or DFL**.

The IoU-based loss measures how well the predicted bounding box overlaps with the ground-truth bounding box.

For example, if the predicted seat-belt box overlaps the actual seat-belt box very well, the localization loss will be smaller. If the predicted box is far away or has the wrong size, the loss will be larger.

DFL helps the model learn more accurate bounding-box coordinates by modeling the distribution of the box coordinates rather than treating them only as single values.

For classification, the model calculates classification loss by comparing the predicted class probabilities with the ground-truth class.

For example, if the actual class is seat belt but the model gives a high probability to another class, the classification loss will increase.

During training, these loss components are combined into the overall detection loss.

The objective of training is to minimize this loss.

So, in simple terms:

Box loss → Is the predicted box in the correct location and size?

DFL → Can the box coordinates be localized more precisely?

Classification loss → Did the model identify the correct class?

The network uses the resulting loss to perform backpropagation and update its weights.

YOU ACHIEVED 90.6% ACCURACY. HOW DID YOU CALCULATE IT?

For my YOLOv8s model, I obtained an accuracy of approximately 90.6% during evaluation.

Since this is an object detection problem, I evaluated the model based on its detection performance rather than treating it as a simple image classification problem.

The model predictions were compared with the ground-truth annotations. A prediction is considered correct based on whether the predicted object and its bounding box sufficiently match the ground-truth object according to the evaluation criteria.

The evaluation results were then used to measure how well the model detected the seat belt classes.

The important point is that in object detection, metrics such as precision, recall, IoU and mAP are generally more appropriate than simple classification accuracy, because the

model has to get both the class and the location of the object correct.

In my project, the YOLOv8s model achieved approximately 90.6% according to the evaluation measure I used.